

RTL Circuit Reference

Thirty circuits, one per page: ten combinational blocks, ten sequential building blocks, and the nine finite-state machines built on the 3-always-block template — plus a page explaining that template itself. Every diagram uses one drawing convention throughout this book:

 clocked register  combinational logic  Mealy output path

Filled boxes hold state and only change on a clock edge; outline boxes are pure combinational logic, settled at all times. In the FSM pages, amber wires mark the one structural difference between a Mealy and a Moore machine — whether a raw input reaches the output logic directly, or only ever passes through the state register first.

CONTENTS

30 Circuits

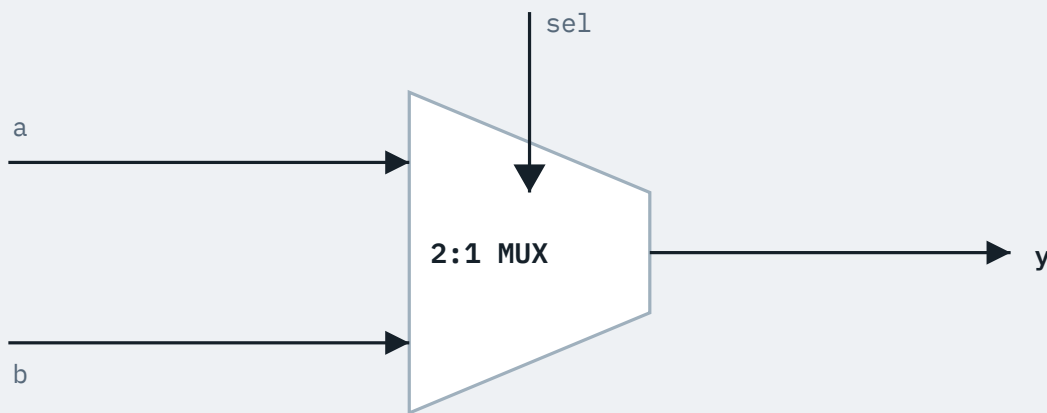
01 Mux2to1	comb
02 PriorityEncoder4to2	comb
03 Decoder3to8	comb
04 Adder4bit	comb
05 ParamALU	comb
06 Comparator4bit	comb
07 ParityGenChecker	comb
08 GrayBinaryConverter	comb
09 BarrelShifter	comb
10 TriStateBusDriver	comb
11 DFlipFlop	seq
12 TFlipFlop / JKFlipFlop	seq
13 EdgeDetector	seq
14 ModNCounter	seq
15 UpDownCounter	seq

16 UniversalShiftReg	seq
17 LfsrPrng	seq
18 RingJohnsonCounter	seq
19 ClkDivider	seq
20 DualPortRam	seq
21 3-Always-Block Template	fsm
22 MealySeqDetector	fsm
23 MooreSeqDetector	fsm
24 TrafficLightController	fsm
25 VendingMachine	fsm
26 SerialAdder	fsm
27 HandshakeFsm	fsm
28 CdcSynchronizer	fsm
29 ArbiterFsm	fsm
30 SafeStateMachine	fsm

Mux2to1

`mux_2to1.v`

What it does: picks one of two inputs, `a` or `b`, and routes it straight to the output `y`, based on the value of `sel`. **How it's coded:** the same circuit results whether you write it as a one-line ternary (`assign y = sel ? b : a;`) or as a `case` statement inside an `always @(*)` block — both synthesize to the same 2:1 multiplexer, a single select line steering one of two data paths through.

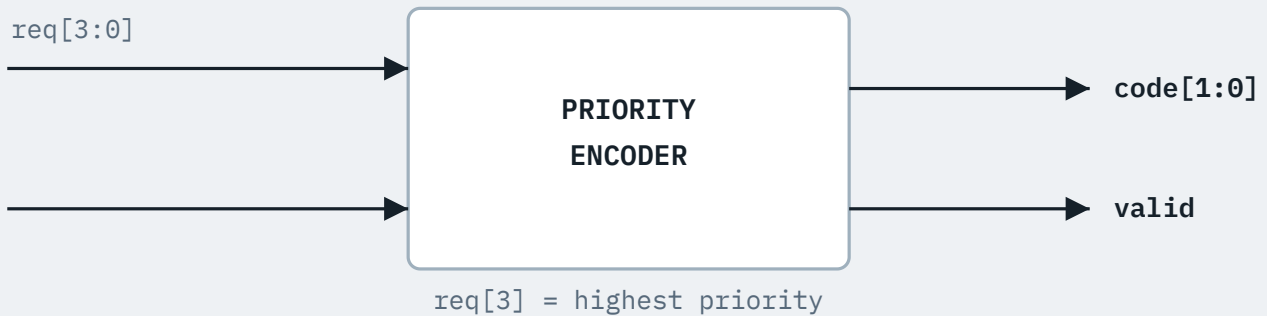


```
in a, b, sel · out y
```

PriorityEncoder4to2

`priority_encoder.v`

What it does: looks at four request lines and reports the index of the highest-priority one that's asserted, as a 2-bit code — convention here is `req[3]` wins over everything else, down to `req[0]` as lowest priority. **Why the valid flag:** a code alone can't tell "req[0] is active" apart from "no request at all", since both would encode as 0 — `valid` resolves that ambiguity for whatever reads the code next.



```
in req[3:0] · out code[1:0], valid
```

Decoder3to8

`decoder_3to8.v`

What it does: the exact inverse of an encoder — turns a 3-bit binary select into one-hot 8-bit output, exactly one of `y[7:0]` going high (the one whose index matches `sel`), the rest staying low. **Why the enable:** with `en` low, every output stays low regardless of `sel`, which is what lets several decoders share one address bus without fighting each other — only the one whose enable is asserted actually drives a line.



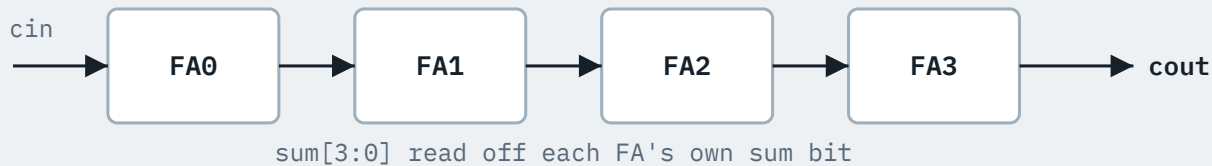
```
in sel[2:0], en • out y[7:0]
```

Adder4bit

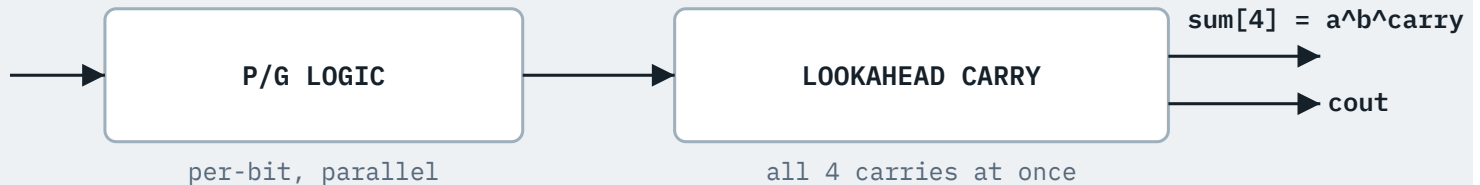
`ripple_carry_adder.v` · `carry_lookahead_adder.v` (bonus)

Ripple carry (top): four 1-bit full-adder cells chained together, each one's carry-out feeding the next one's carry-in — simple, but `sum[3]` can't be correct until the carry has rippled all the way from `cin` through three prior stages, so worst-case delay grows with width. **Carry lookahead (bottom):** computes propagate/generate signals for every bit in parallel, then a lookahead block derives all four carries at once from those — more logic, but a flat delay regardless of width.

carry ripples FA0 → FA3, one full-adder delay per bit



`a[4]`, `b[4]`, `cin`

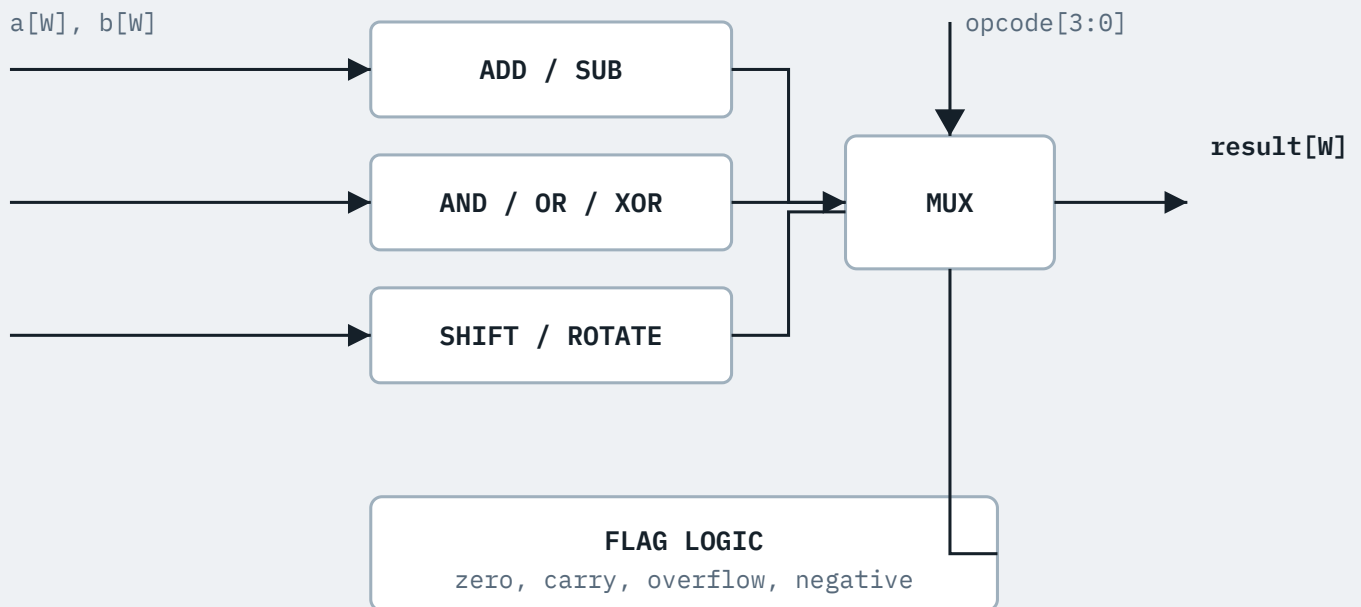


`in` `a[4]`, `b[4]`, `cin` · `out` `sum[4]`, `cout` · two implementations, one interface

ParamALU

`alu.v`

What it does: every function unit — add/sub, the bitwise logic ops, shift/rotate — computes its own result on every input change, all in parallel; `opcode` then just steers a mux to pick which one actually reaches `result`. Nothing is "run" selectively; the unused units' outputs are simply discarded. **Flags:** derived from whichever result the mux selected, so `zero` / `carry` / `overflow` / `negative` always describe the operation that was actually requested.

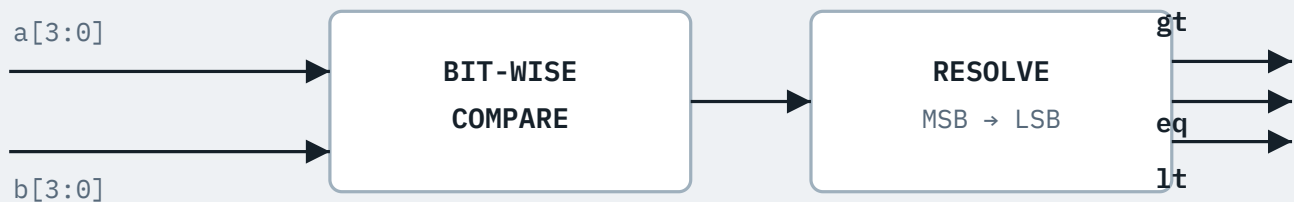


in `a[W]`, `b[W]`, `opcode[N]` · **out** `result[W]`, `zero`, `carry`, `overflow`, `negative`

Comparator4bit

`comparator_4bit.v`

What it does: compares two 4-bit values and asserts exactly one of `gt` / `eq` / `lt`. **How:** a first stage compares each bit pair independently; a second stage resolves those bit results starting from the MSB — the highest bit where `a` and `b` disagree decides the whole comparison, exactly like comparing two numbers by hand, most-significant digit first.



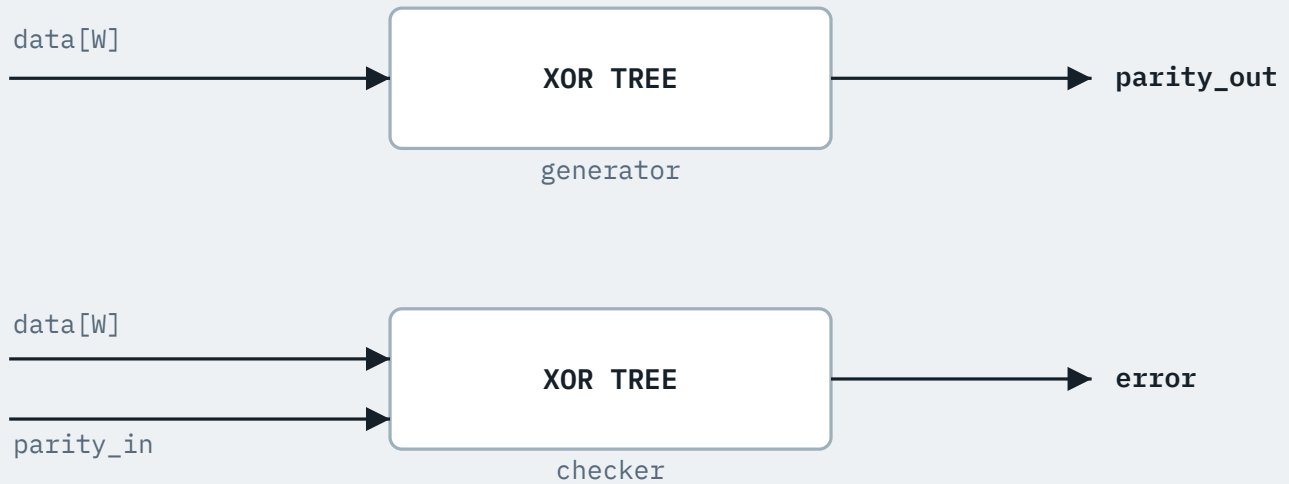
in `a[4]`, `b[4]` · **out** `gt`, `eq`, `lt`

ParityGenChecker

parity.v

Generator: XORs every bit of `data` together into a single `parity_out` bit — a running "is the number of 1s odd?" check, attached to the data before it's sent or stored.

Checker: re-runs the same XOR tree on the received `data` plus the received `parity_in`; if nothing flipped in transit, the result is always 0 — any single-bit flip makes it 1, raising `error`.

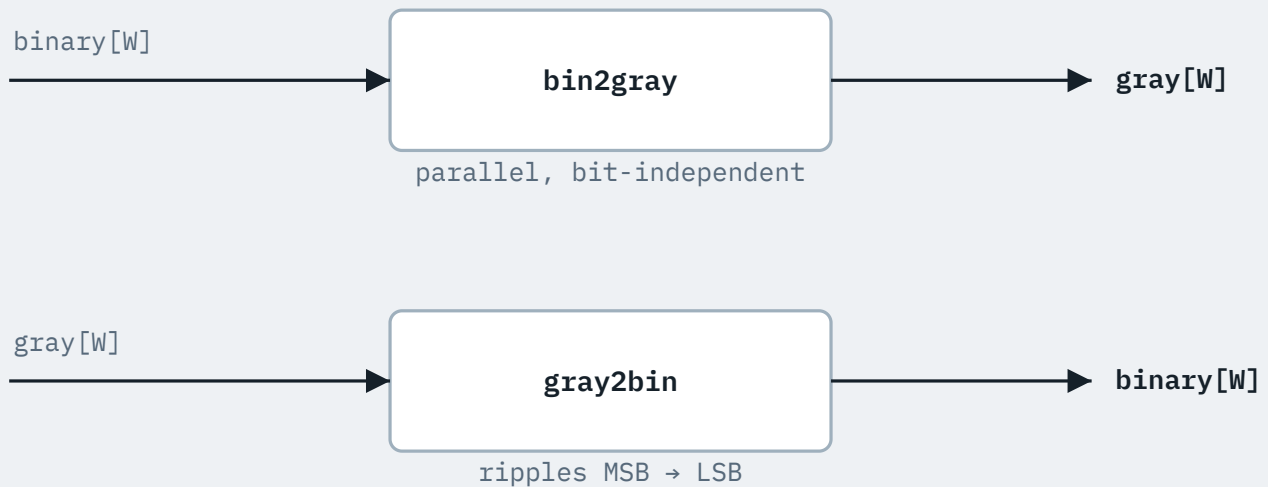


gen: `data[W] → parity_out` · **check:** `data[W], parity_in → error`

GrayBinaryConverter

`gray_binary.v`

bin2gray: $\text{gray}[i] = \text{binary}[i] \oplus \text{binary}[i+1]$ for every bit, all independent of each other — a flat, single-gate-delay conversion, which is why Gray code is cheap to generate on the fly (e.g. for a counter whose output crosses clock domains, where only one bit should ever change per step). **gray2bin:** the inverse needs the MSB of `binary` first (it equals the MSB of `gray` outright), then ripples $\text{binary}[i] = \text{gray}[i] \oplus \text{binary}[i+1]$ down from there — the one direction that isn't fully parallel.



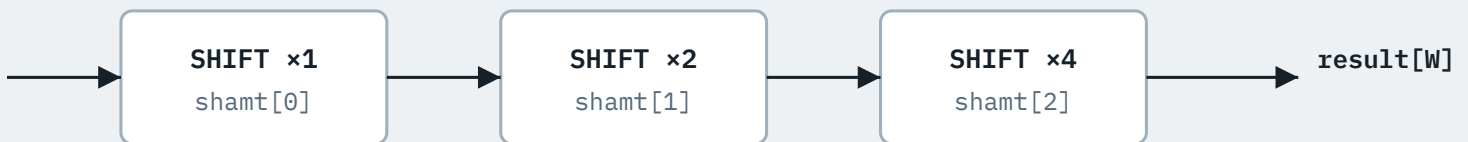
bin2gray: `binary[W] → gray[W]` · **gray2bin:** `gray[W] → binary[W]`

BarrelShifter

```
barrel_shifter.v
```

What it does: shifts `data` by any amount in one pass, using $\log_2(W)$ cascaded stages — one stage per bit of `shamt`, each stage either passing data straight through or shifting it by its own power-of-2 amount (1, 2, 4, ...), so any combination of stages firing adds up to the requested total shift. **logical vs. arithmetic:** both fill the vacated bits the same way on a left shift (zeros); on a right shift, logical mode fills with zeros while arithmetic mode fills with copies of the original sign bit, preserving a signed value's sign.

`data[W], mode`



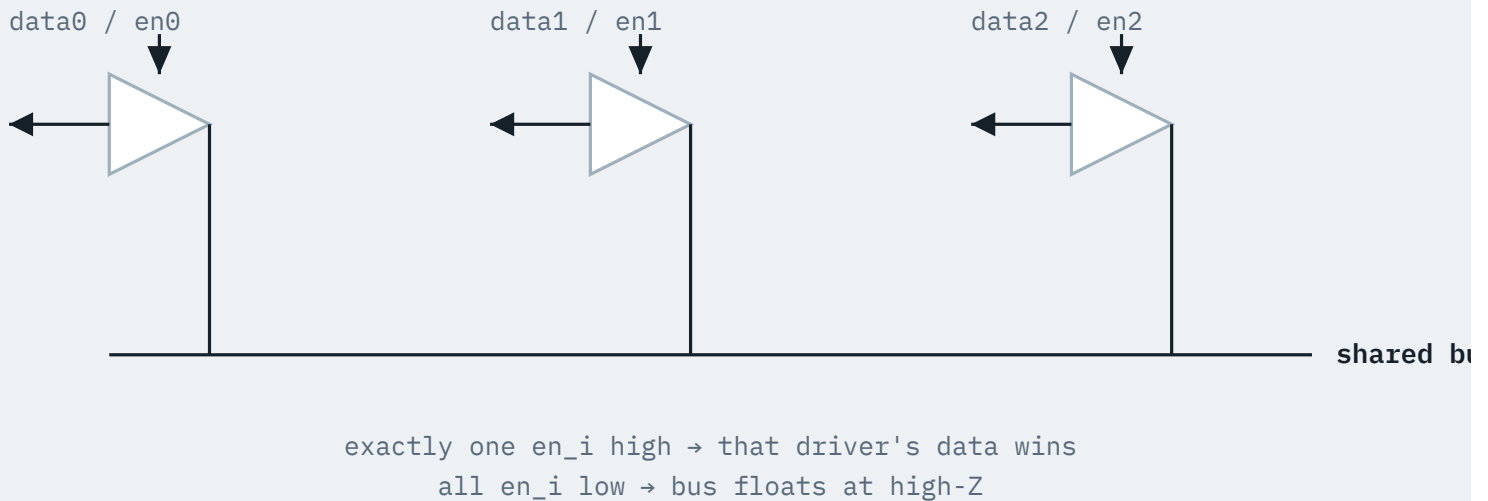
logical: zero-fill both directions
 arithmetic: sign-extend fill on right shift

```
in data[W], shamt[log2 W], mode · out result[W]
```

TriStateBusDriver

```
tristate_buffer.v
```

What it does: each buffer drives its `data` onto a shared bus only while its own `enable` is high; the moment `enable` drops, that driver disconnects entirely and presents high-impedance (Z) instead of a 0 or a 1. **Why it matters:** this is what lets several devices share one physical bus line — a memory-mapped peripheral bus, for instance — as long as the surrounding logic guarantees at most one enable is ever high at a time; two drivers enabled together would fight each other electrically.

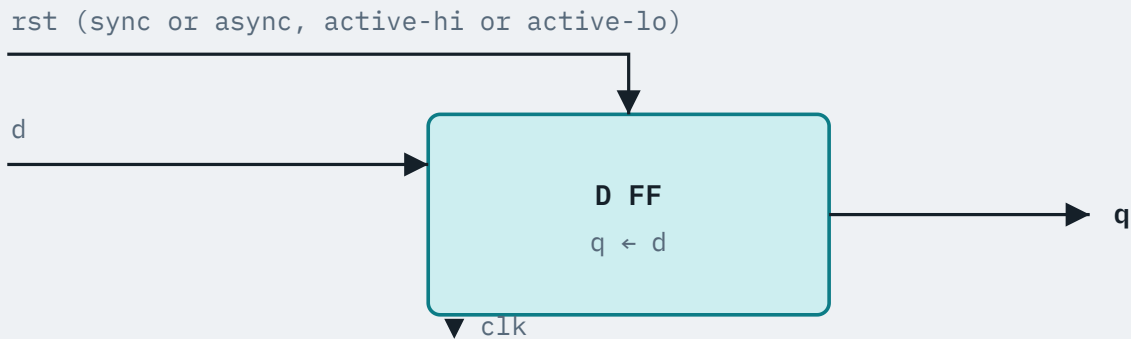


each driver: data, enable → shared bus (high-Z when disabled)

DFlipFlop

d_ff.v · 4 variants

What it does: the atom every register in this book is built from — on each rising clock edge, `q` takes on whatever `d` was. **The four variants:** reset can be written synchronous (`if(rst)` inside `always @(posedge clk)`), so it only takes effect on a clock edge) or asynchronous (`rst` added to the sensitivity list itself, so it forces `q` immediately, clock or no clock); independently, the reset can be active-high or active-low. Same register, same diagram — only the reset path's polarity and timing change.



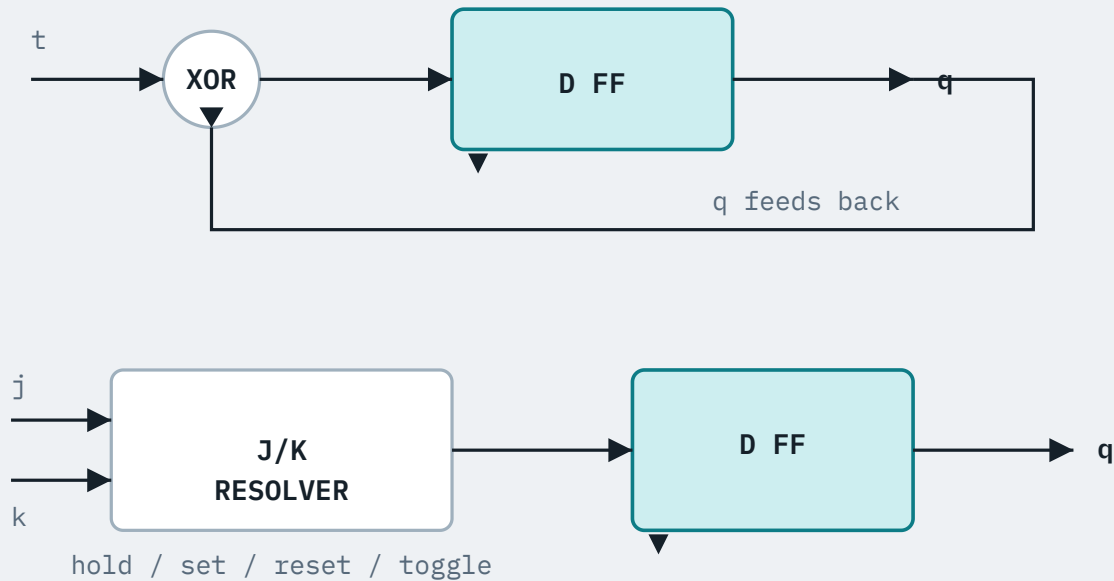
sync: rst only checked on a clock edge
 async: rst in the sensitivity list, acts immediately

in clk, d, rst (or arst) · **out** q · all 4 variants share this diagram

TFlipFlopJKFlipFlop

t_ff.v · jk_ff.v

T flip-flop (top): toggles `q` every clock when `t` is high, and holds when `t` is low — built by XOR-ing the current `q` back against `t` and feeding that into an ordinary D flip-flop. **JK flip-flop (bottom):** the classic 4-way table (00=hold, 01=reset, 10=set, 11=toggle) resolved by a small combinational block into a single D input — both variants are really just a D-FF with a bit of logic deciding what to feed it.



T-FF in clk, t · JK-FF in clk, j, k · both out q

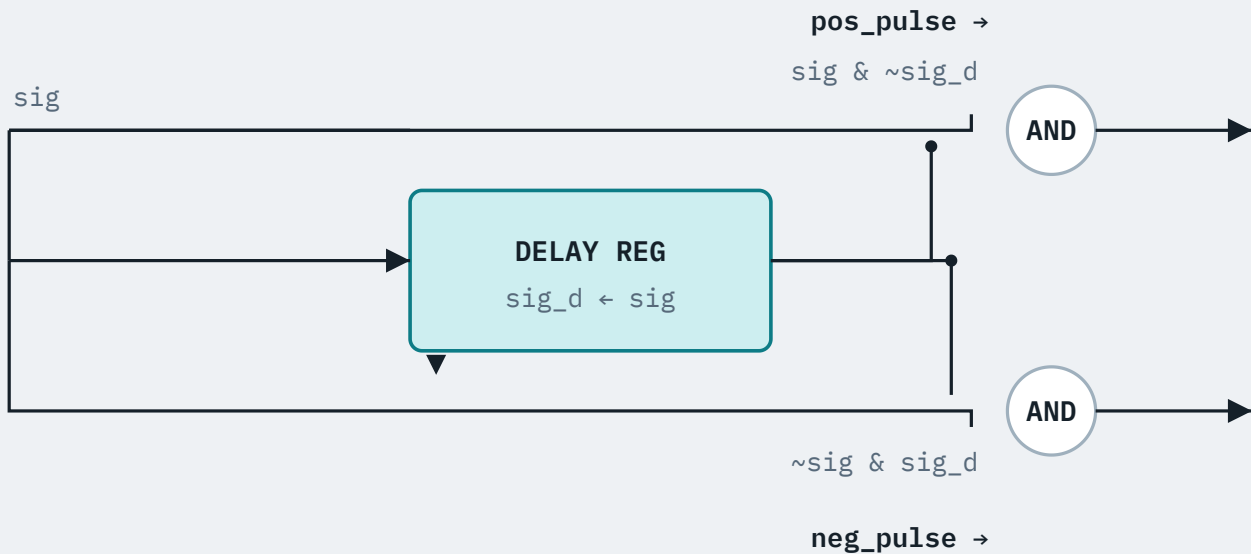
EdgeDetector

`edge_detector.v`

What it does: turns a level signal into a single clock-wide pulse the instant it changes.

How: one register samples `sig` every clock into `sig_d` — one cycle behind.

Comparing the two catches the transition: `sig & ~sig_d` is high for exactly one cycle right after a rising edge, and `~sig & sig_d` does the same for a falling edge. This is the standard way to convert a button press or a slow external signal into a clean, one-shot pulse an FSM can react to.

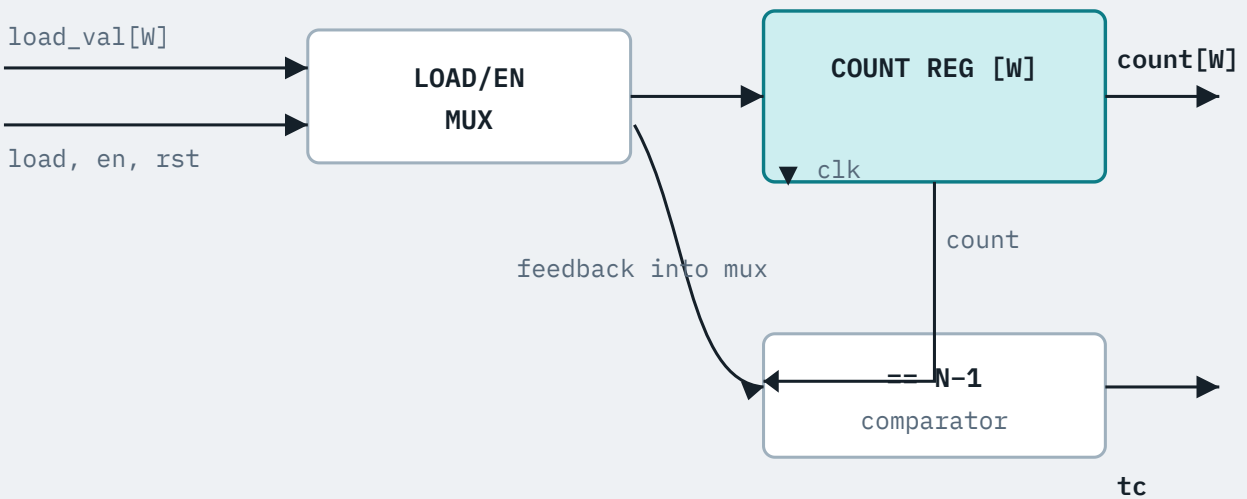


in `clk, rst, sig` · **out** `pos_pulse, neg_pulse` (each 1 cycle wide)

ModNCounter

`mod_n_counter.v`

What it does: counts 0 up to $N-1$ and wraps back to 0, instead of a plain binary counter's power-of-2 wrap. **Priority:** a load/enable mux sits in front of the register — synchronous `load` overrides whatever the counter would otherwise do, and only when neither loading nor disabled does the counter actually advance. `tc` (terminal count) is a plain comparator watching for $N-1$, useful for chaining counters or flagging a full cycle to the rest of the design.

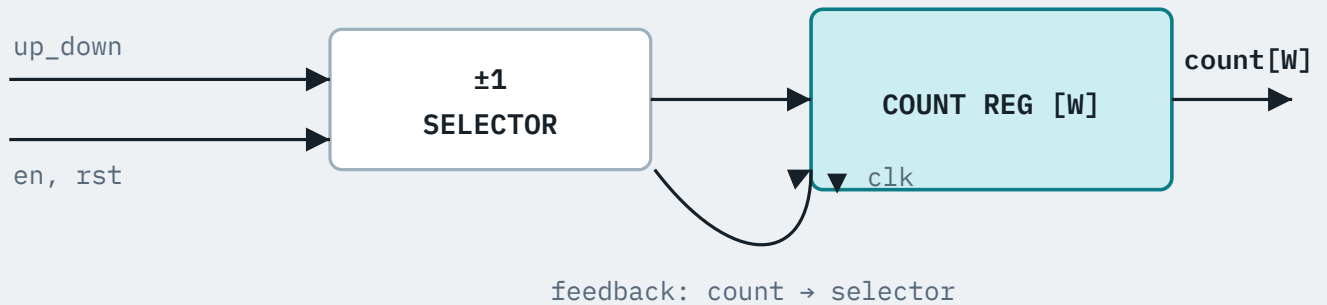


in `clk, rst, en, load, load_val[W]` · **out** `count[W], tc`

UpDownCounter

updown_counter.v

What it does: a free-running binary counter where `up_down` picks the direction each cycle — a single selector chooses between `count+1` and `count-1` going back into the same register. **Wraparound:** no special-case logic for the ends; ordinary binary overflow/underflow does the wrapping for you (counting up past all-1s rolls to all-0s, counting down past all-0s rolls to all-1s), which is exactly the two's-complement behavior of `+` / `-` in hardware.

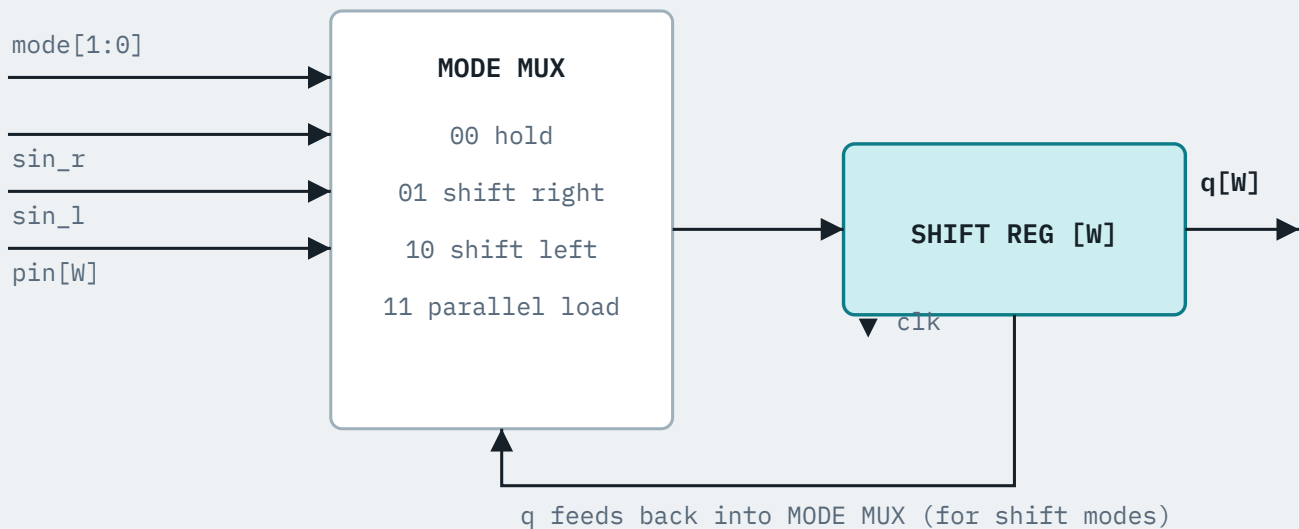


```
in clk, rst, en, up_down · out count[W]
```

UniversalShiftReg

universal_shift_reg.v

What it does: one register, four classic shift-register topologies, selected purely by how you drive `mode` and which pins you watch. Hold it in shift-right mode and read the single `q[0]` bit each cycle and you have a **SISO**; do the same but read the whole bus after W shifts and you have a **SIPO**. Parallel-load once (**PIPO**), then switch to shift mode and read the serial output bit as it drains, and you have a **PISO**. Same silicon, four names, purely a function of how the surrounding design uses it.

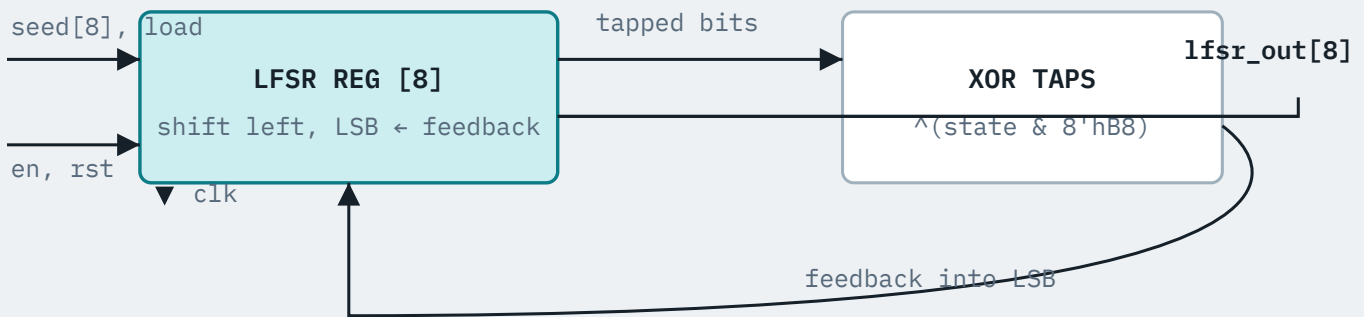


```
in clk, rst, mode[1:0], sin_r, sin_l, pin[W] · out q[W]
```

LfsrPrng

lfsr_prng.v

What it does: an 8-bit Fibonacci LFSR — each cycle it shifts left by one, and the bit shifted into the LSB is the XOR of a fixed set of "tapped" bit positions rather than a fresh input. With the right tap positions (here `8'hB8`, corresponding to the polynomial $x^8 + x^6 + x^5 + x^4 + 1$), this cycles through all 255 non-zero 8-bit states before repeating — a maximal-length sequence, cheap pseudo-random bits from almost no logic. **The all-zero guard:** an all-zero state would XOR to another all-zero state forever, so reset and load both force a non-zero seed.

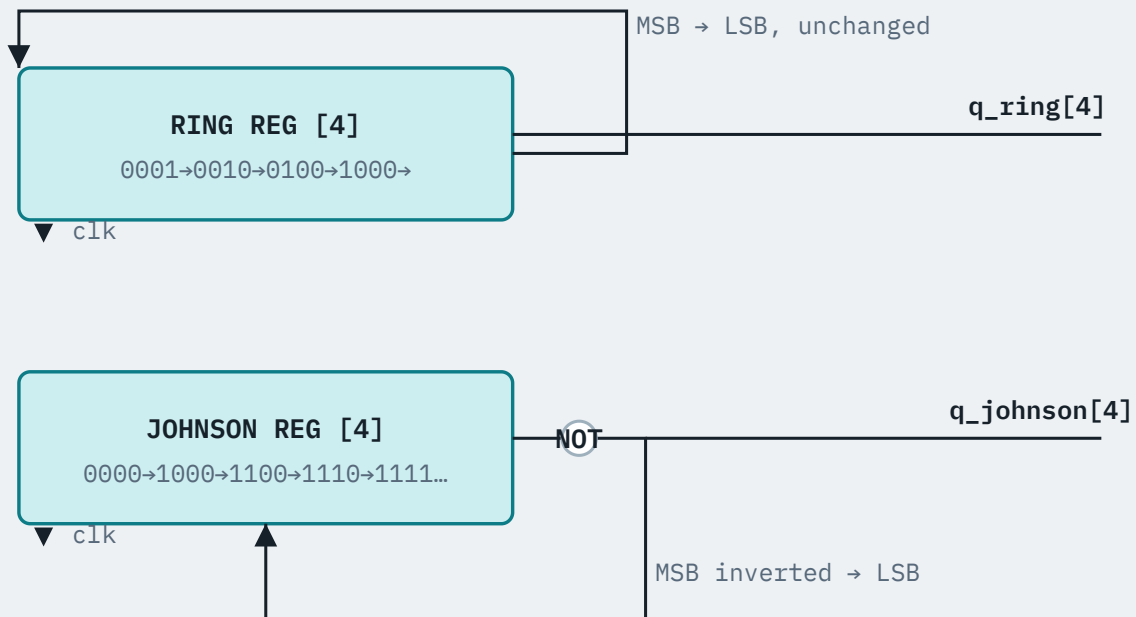


in `clk, rst, en, load, seed[8]` · **out** `lfsr_out[8]` · period 255 states

RingJohnsonCounter

```
ring_johnson_counter.v
```

Ring counter (top): a shift register whose MSB feeds straight back into the LSB unchanged. Starting from `0001` it cycles `0001→0010→0100→1000→0001...` — always exactly one hot bit, period = width. **Johnson counter (bottom):** identical idea, but the fed-back bit is inverted first, which doubles the period to $2 \times \text{width}$ and produces a smooth, evenly-spaced count sequence (`0000→1000→1100→1110→1111→0111...`) useful for generating low-skew multi-phase clock-like signals.

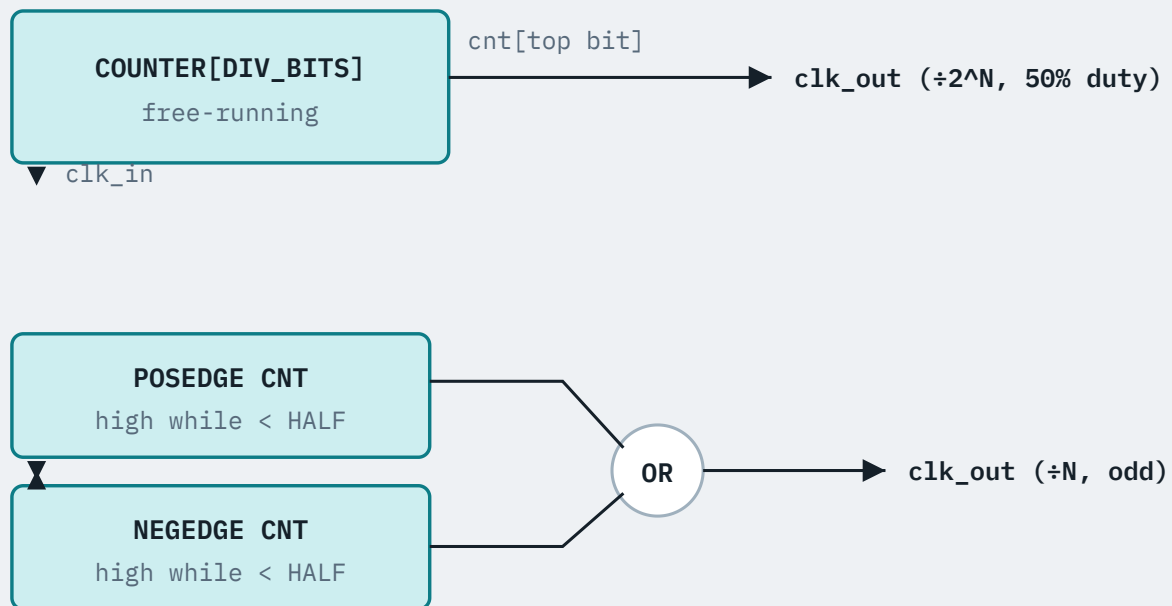


```
in clk, rst (both) · out q_ring[4], q_johnson[4]
```

ClkDivider

clk_divider.v

Power-of-2 (top): a free-running counter's own top bit already toggles at exactly $1/2^n$ the input rate with a perfect 50% duty cycle — no extra logic needed, just tap the right bit. **Odd integer (bottom):** can't be done that cleanly, since an odd division can't split into two exactly equal halves. The standard fix is two counters, one clocked on the input's rising edge and one on its falling edge, each held high for the first half of its own count; OR-ing them together gives a period of exactly N input cycles with as close to 50% duty as an odd N allows.

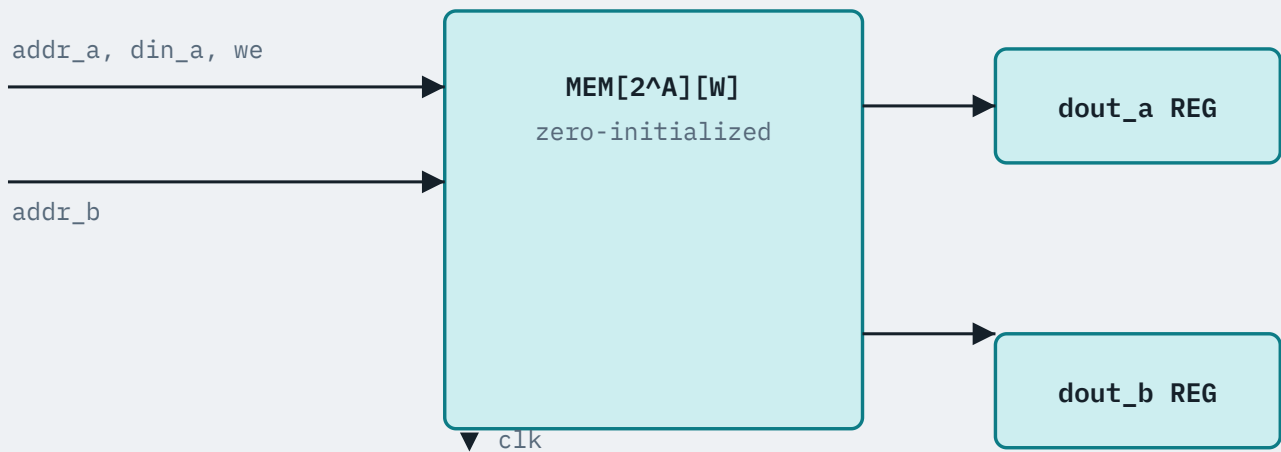


in clk_in, rst • **out** clk_out • two independent modules, one file

DualPortRam

dual_port_ram.v

What it does: one write port, two independent read ports, all sharing the same memory array and clock. Port A can write and read at the same address in the same cycle; port B only ever reads, completely independently of what A is doing — a common shape for a register file or a simple dual-access scratchpad. **Read-during-write:** when A writes and reads the same address in the same cycle, `dout_a` returns the *old* value, not the one just written — standard block-RAM "read-old-data" behavior, matching what real FPGA BRAM primitives do.

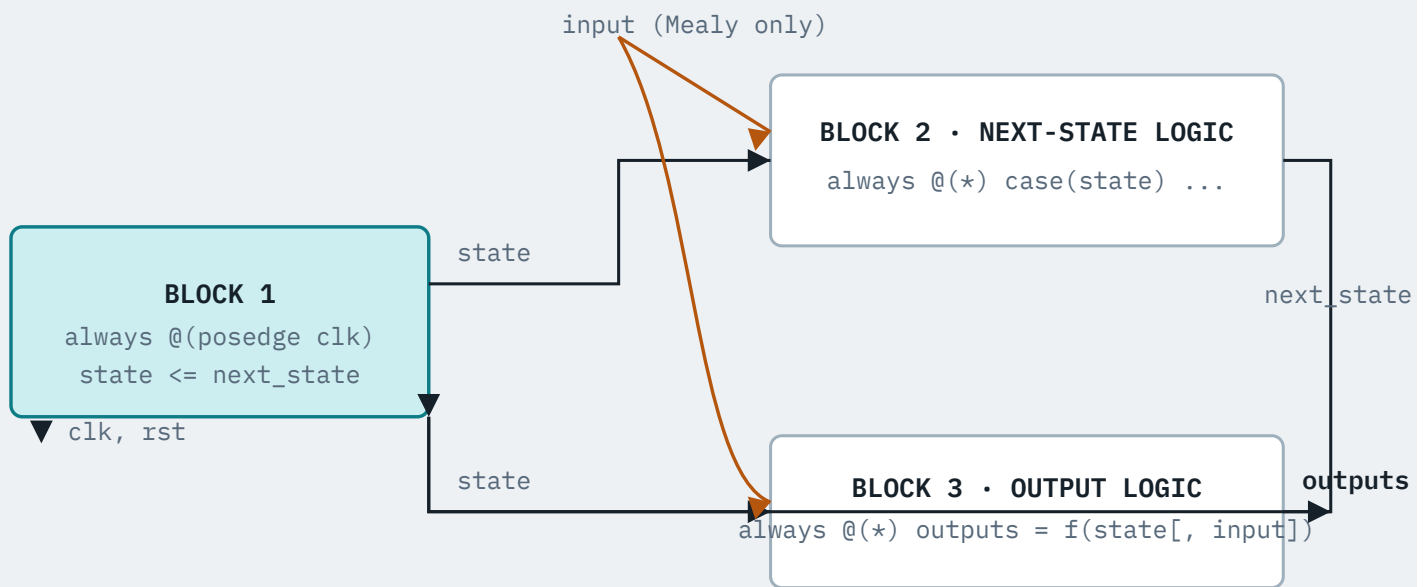


in `clk, we, addr_a, addr_b, din_a` • **out** `dout_a, dout_b`

3-Always-Block FSM Template

the pattern, not a project

The skeleton every FSM on the next nine pages is built from. **Block 1** is the *only* place state actually changes — a plain clocked register, `state <= next_state`. **Block 2** is a combinational `case` statement that decides what `next_state` should become, purely from the current `state` (and, for many FSMs, an input). **Block 3** is a second combinational block that turns state into outputs — and it never writes state back; outputs are a read-only view of the FSM, never a side channel for its logic. Splitting the design into exactly these three always blocks, rather than mixing state update and output logic together, is what keeps a hand-written FSM synthesizable, readable, and free of accidental latches.

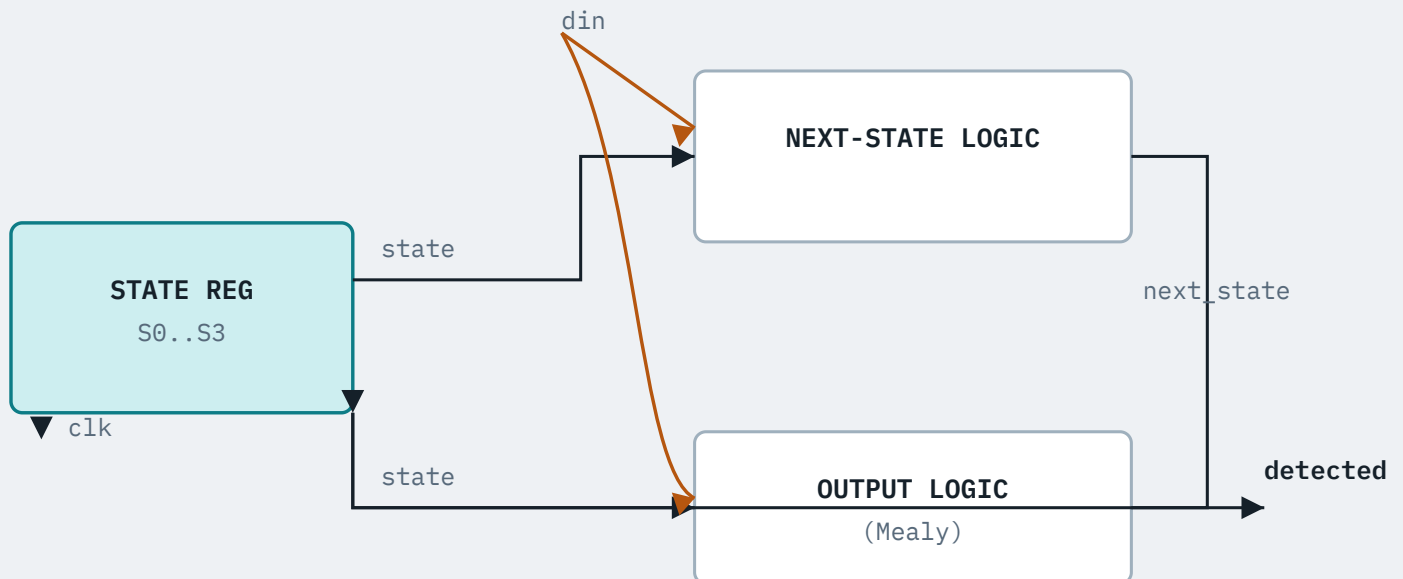


Same three blocks in every FSM page that follows · only what feeds Block 3 — state alone (Moore), or state + a raw input (Mealy) — changes between them.

MealySeqDetector

```
mealy_seq_detector.v
```

What it does: watches a serial bit stream and pulses `detected` the instant it sees the pattern "1001", tracking progress through 4 states (matched nothing / "1" / "10" / "100"). **Mealy, specifically:** `din` reaches Block 3 (output logic) directly, alongside `state` — so `detected` can go high in the very same cycle the last matching bit arrives, one cycle earlier than an equivalent Moore machine could manage. **OVERLAP parameter:** after a match, overlapping mode treats the just-seen "1" as a possible start of the next match (so "1001001" finds two matches); non-overlapping mode discards it and restarts the search from scratch.

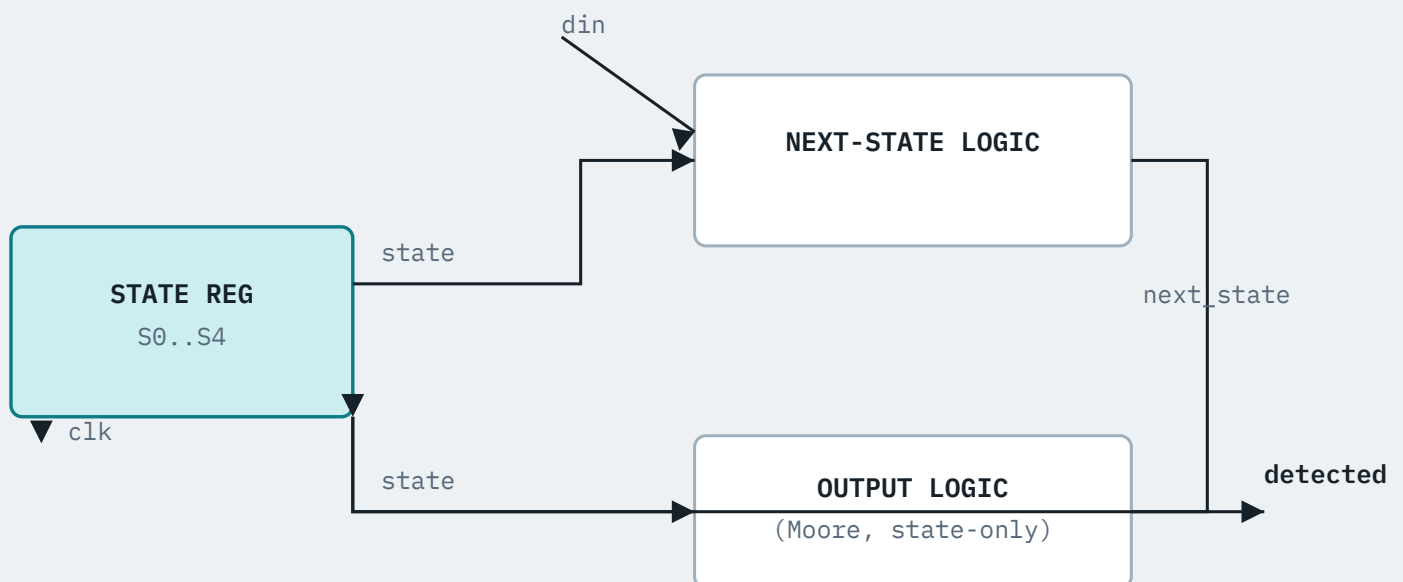


```
in clk, rst, din · out detected · param OVERLAP
```

MooreSeqDetector

moore_seq_detector.v

What it does: the same "1001" detector, restructured as a Moore machine. **The difference from Mealy:** a Moore output can only ever be a function of `state`, never of a raw input directly — so this needs a fifth state, S4, purely to represent "just matched" as its own distinct state. `detected` is asserted for a full clock cycle whenever `state == S4`, one cycle later than the Mealy version's pulse, but glitch-free and unconditionally stable for that whole cycle since it depends on nothing but the registered state.

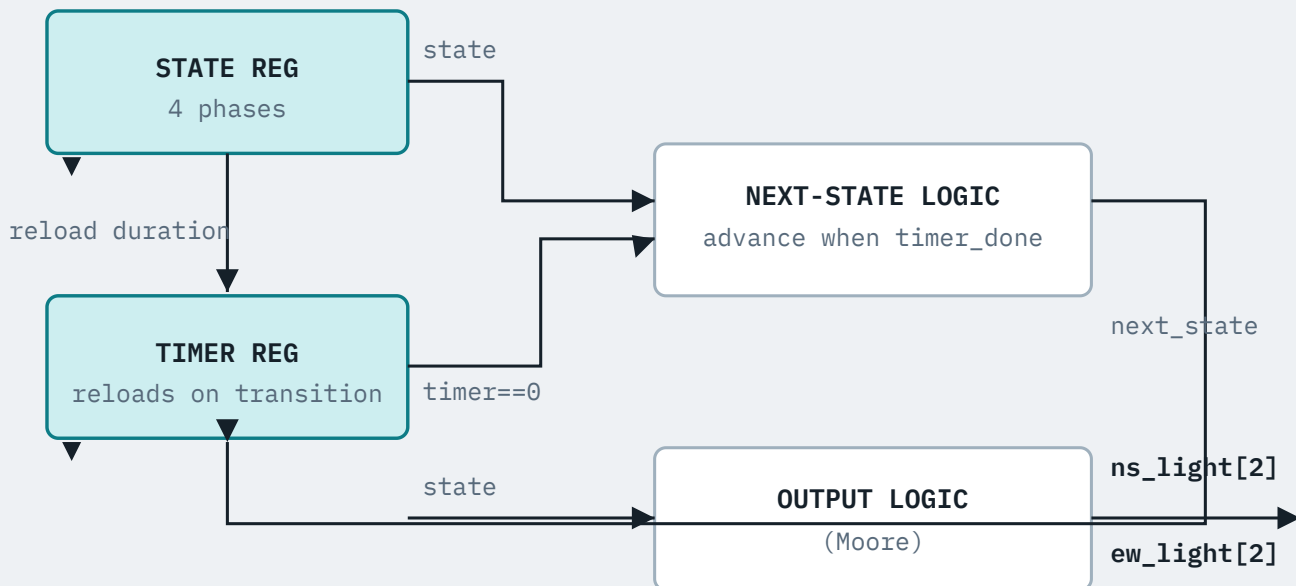


in `clk, rst, din` · **out** `detected` · **param** `OVERLAP`

TrafficLightController

traffic_light_fsm.v

What it does: cycles four phases — NS green, NS yellow, EW green, EW yellow — each held for a parameterized number of cycles (`GREEN_TIME` , `YELLOW_TIME`). **The timer register:** a second sequential block alongside the state register; it reloads to the new phase's duration on every transition, and otherwise just counts down. Block 2 (next-state logic) only advances the FSM when it sees `timer_done` , so the timer is really acting as a guard condition feeding the same next-state decision every other FSM in this book makes from state and input alone.

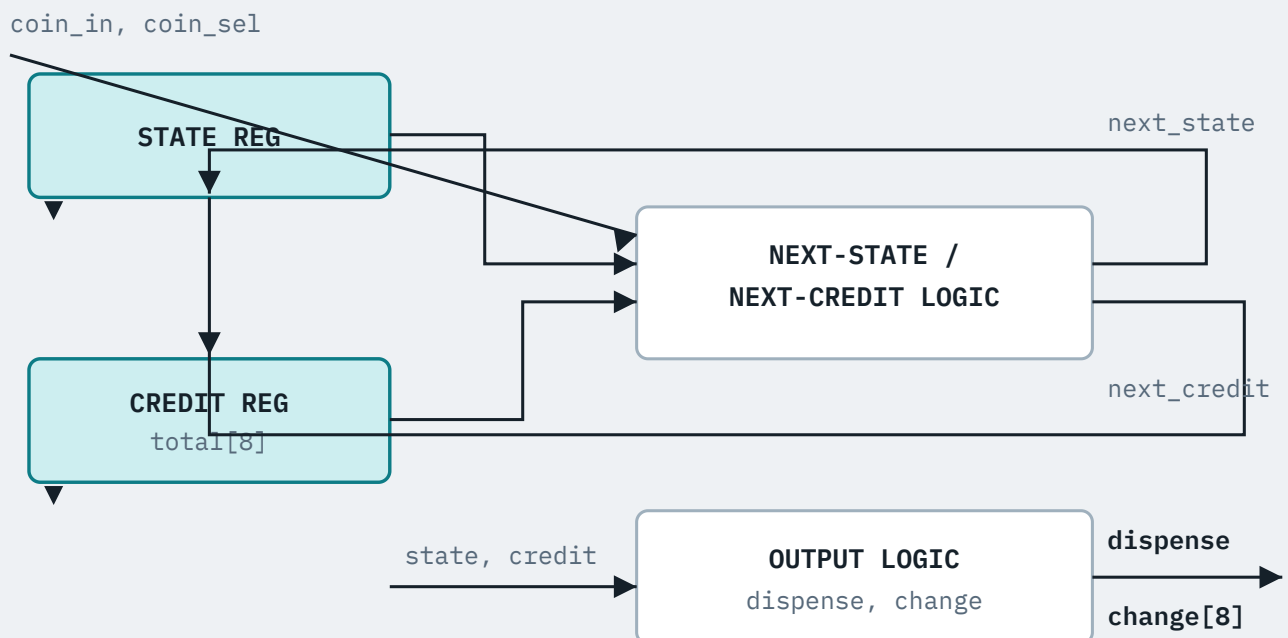


```
in clk, rst · out ns_light[2], ew_light[2] · param GREEN_TIME, YELLOW_TIME
```

VendingMachine

vending_machine.v

What it does: just two states, IDLE and DISPENSE, plus a credit register that accumulates coin value (5/10/25¢ selected by `coin_sel`) while idle. **Why next-state and next-credit are computed together:** whether to move to DISPENSE depends on whether *this cycle's* coin pushes the running total to `PRICE` or beyond — both decisions read the same `total + coin_val` sum, so Block 2 naturally produces `next_state` and `next_credit` side by side. DISPENSE itself lasts exactly one cycle: it asserts `dispense` and `change = total - PRICE`, resets credit to 0, and returns to IDLE.

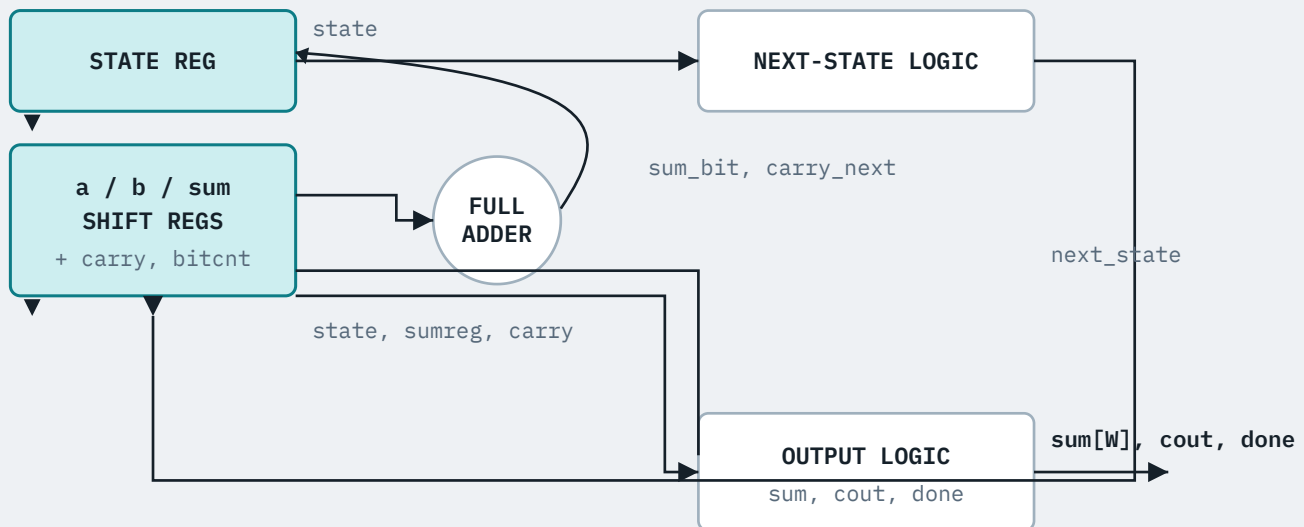


```
in clk, rst, coin_in, coin_sel[2] · out dispense, change[8], credit[8] · param PRICE
```

SerialAdder

serial_adder.v

What it does: adds two W -bit numbers one bit at a time through a single 1-bit full adder, instead of W full adders at once — a classic area-for-time tradeoff. IDLE loads both operands and clears the carry on `start`; ADD runs for exactly W cycles, each one consuming the current LSB of both shift registers, computing `sum_bit` and `carry_next`, then shifting everything right by one and shifting `sum_bit` into the result register's MSB; DONE holds the finished `sum`, `cout`, and a `done` flag for exactly one cycle before returning to IDLE.

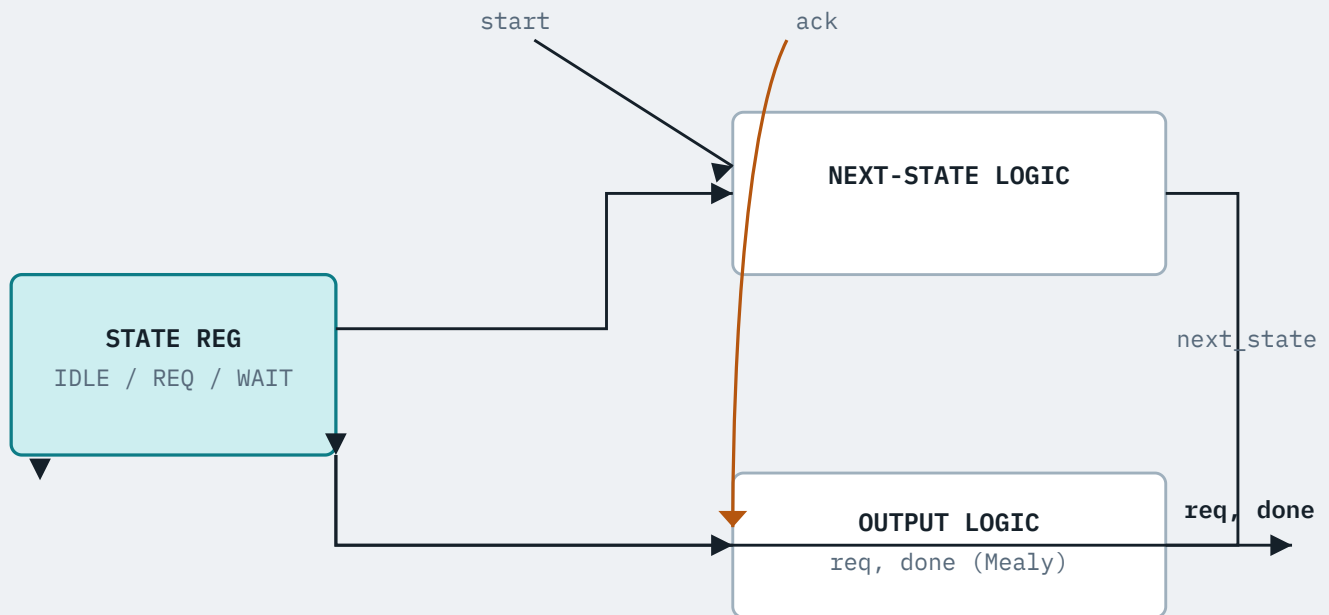


```
in clk, rst, start, a[W], b[W] · out sum[W], cout, done · param WIDTH
```

HandshakeFsm

handshake_fsm.v

What it does: drives one side of a classic 4-phase req/ack handshake. IDLE waits for `start`; REQ_HIGH asserts `req` and waits for the receiver's `ack`; REQ_LOW_WAIT drops `req` again and waits for `ack` to fall back to 0, confirming the receiver saw the transaction end. **Why it's Mealy:** `done` needs to pulse the exact cycle `ack` is observed low while still in REQ_LOW_WAIT — that's `state AND ack` together, which only Block 3 reading a raw input directly (not just registered state) can produce without an extra cycle of delay.

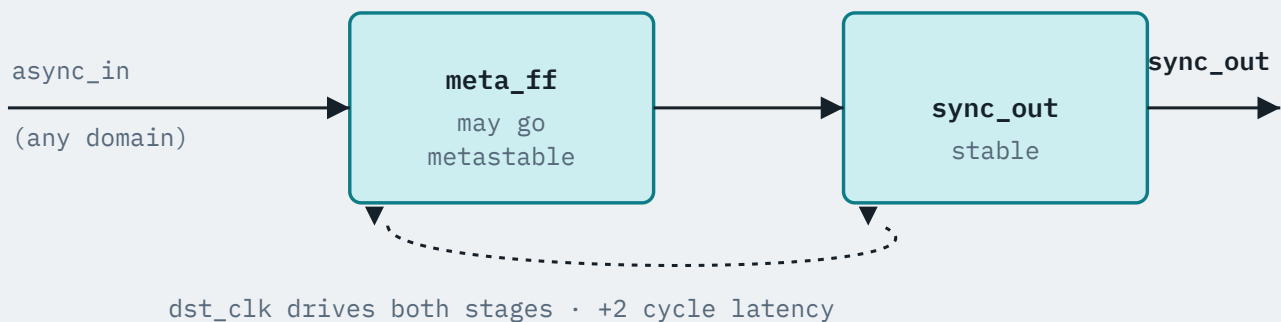


in clk, rst, start, ack · **out** req, done

CdcSynchronizer

cdc_sync.v

What it does: not a control FSM at all — grouped here because it's the primitive every FSM needs the moment it has to react to a signal from another clock domain. Two flip-flops in series, both clocked by `dst_clk`: the first (`meta_ff`) is the one that might actually sample `async_in` mid-transition and go metastable; by the time the second stage captures `meta_ff`, there's been a full clock period for any metastability to resolve, so `sync_out` is statistically safe to use. **The cost:** 2 cycles of added latency, and the guarantee only holds if nothing downstream tries to read `meta_ff` directly.

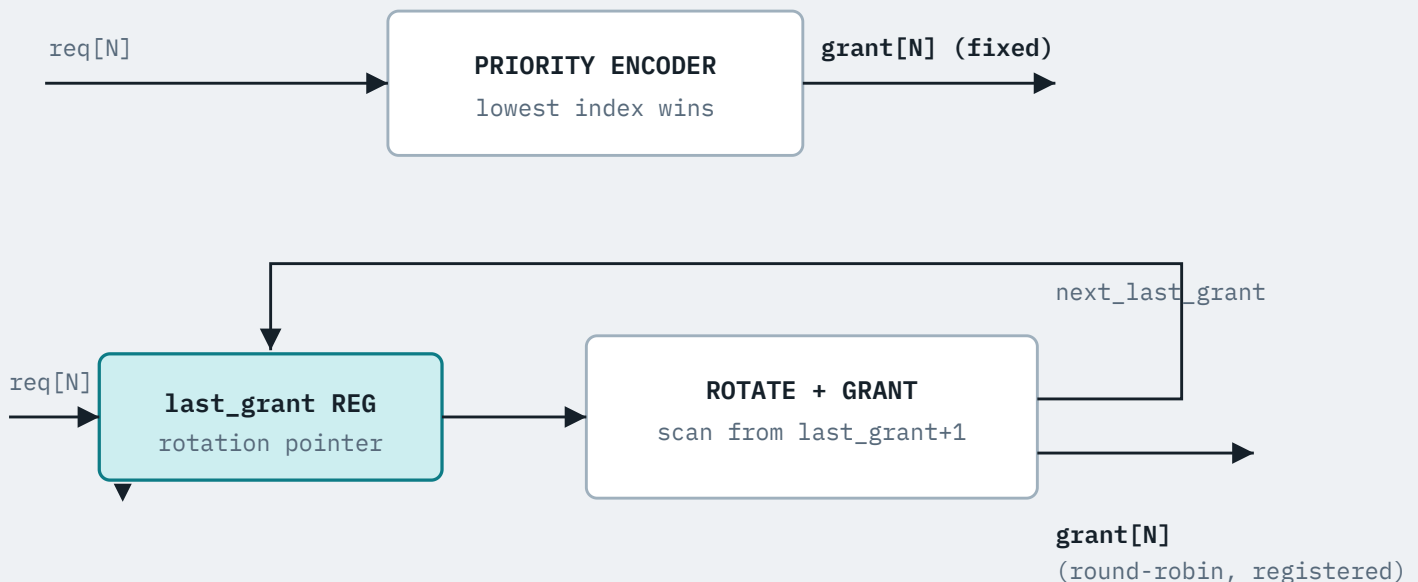


in `dst_clk`, `rst`, `async_in[W]` · **out** `sync_out[W]`

ArbiterFsm

arbiter.v

Fixed priority (top): purely combinational — no state at all. A priority encoder just grants the lowest-indexed requester that's currently asking; simple and predictable, but a low-index requester that asks constantly can starve everyone else. **Round robin (bottom):** adds exactly one register, `last_grant`, holding the index most recently served. Each cycle, the grant logic scans starting right after `last_grant` rather than always from 0, so priority rotates and every active requester eventually gets served — that single register is the entire difference between "fast but unfair" and "fair" arbitration here.

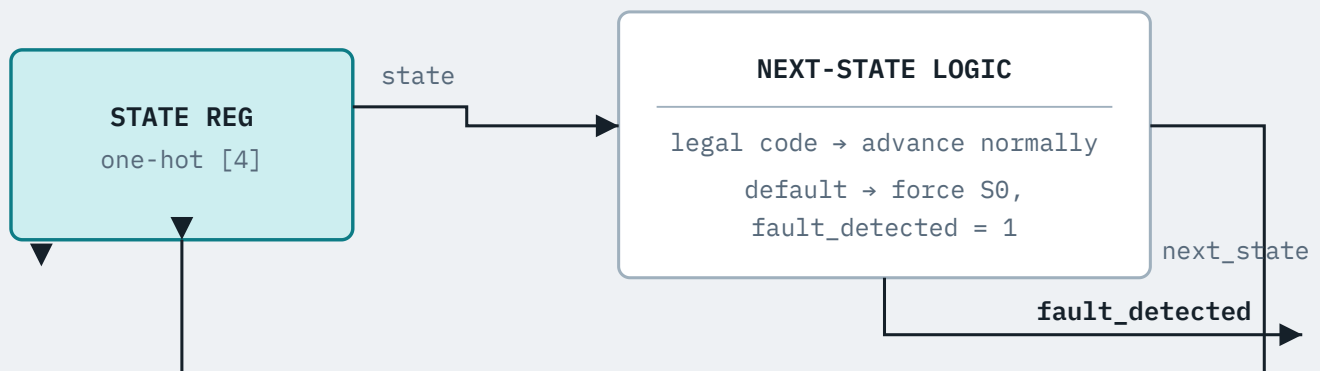


fixed: `req[N] → grant[N]` · **round-robin:** `clk, rst, req[N] → grant[N]`

SafeStateMachine

safe_fsm.v

What it does: a plain 4-state one-hot FSM ($S0 \rightarrow S1 \rightarrow S2 \rightarrow S3 \rightarrow S0$ while `go` is held) with one addition: Block 2's `case` statement has a `default` arm covering every state code that *isn't* one of the four legal one-hot values. **Why it matters:** a one-hot register has far more illegal codes than legal ones (16 possible 4-bit values, only 4 legal) — if radiation, a power glitch, or a simulation/synthesis mismatch ever flips the register into one of those illegal codes, the default arm catches it, forces `next_state` back to `S0`, and raises `fault_detected` for that one cycle. This "safe FSM" pattern is standard practice for one-hot designs precisely because their encoding leaves so much room for undetected corruption otherwise.



in `clk, rst, go` · **out** `state_out[4]` (one-hot), `fault_detected`

